

INTERVIEW PREPARATION SERIES

Python Fundamentals

Production War Stories

15 Scenario-Based Questions from Realistic Engineering Contexts

ABOUT THIS DOCUMENT

15 scenario-based questions on Python fundamentals — each set in a realistic, messy, production-like context. Designed to make engineers with ~3 years of experience think deeply about why fundamentals matter under real-world pressure.

SCENARIO 01

The Mutable Default That Followed Users Home

A SaaS platform has a `create_user` helper: `def create_user(name, roles=[]): ... roles.append("member")`. After deploying, every newly-created user mysteriously inherits the roles of users created before them. The first user has `["member"]`; the second has `["member", "member"]`; the hundredth has the list of “member” a hundred times. Some users created in admin flows even inherit `"admin"` because an earlier admin creation appended it. Security treats it as a critical incident.

QUESTION

Walk me through exactly what’s happening. Why is this a Python-specific bug, and how do you prevent the entire class of bug in your codebase?

EXPECTED DEPTH

- Default arguments are evaluated *once*, at function-definition time — not at each call. The `[]` in `roles=[]` creates one list object that becomes the default forever.

- Every call that doesn't pass an explicit `roles` gets that same list. `.append("member")` mutates it. The next caller sees the mutation. Every user shares state via the function's default.
- This is the most famous Python gotcha. Applies to any mutable default: lists, dicts, sets, custom objects. Doesn't apply to immutables (`int`, `str`, `tuple`, `None`, `frozenset`).
- Fix:

```
def create_user(name, roles=None):  
    if roles is None:  
        roles = []  
    roles.append("member")  
    return roles
```

- Prevent at scale: enable `ruff` (or `pylint/flake8-bugbear`) and turn on the `B006` rule ("do not use mutable data structures for argument defaults"). Pre-commit hook. Treat as error. Now this can't merge.
- Lesson: any bug class with a clear lint rule should be a lint failure, not a code-review failure. Reviewers miss things; CI doesn't.

SCENARIO 02

The Encoding Crash That Only Happened in Production

A data import tool reads CSV files from customers. The dev team tests with sample files; everything works. A new customer uploads a file containing French names like "François" and "Desée." The import crashes with `UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7 in position 47`. The team's code is `with open(path) as f: data = f.read()`. They had no encoding-related symptoms in two years of operation.

QUESTION

Why did this break now? Walk through the encoding chain, and tell me what robust file-reading code looks like for a multi-customer environment.

EXPECTED DEPTH

- `open()` in text mode (the default) decodes bytes using the platform's preferred encoding. On modern Linux/macOS, that's UTF-8. On Windows, that's often `cp1252` or `cp1250`. The customer's file isn't UTF-8.
- Likely encoding: Windows-1252 or Latin-1 — common when CSVs are exported from Excel on a Windows machine. The byte `0xe7` is "ç" in those encodings, but UTF-8 expects two bytes for that character.
- Why no symptoms before: previous customers happened to send ASCII-only or UTF-8 files. Encoding bugs are silent until they hit a byte that means different things in different encodings.
- Fix 1: be explicit. `open(path, encoding="utf-8")` — deterministic across platforms, but still fails on non-UTF-8 input.
- Fix 2: be tolerant. `open(path, encoding="utf-8", errors="replace")` — unparseable bytes become `?`. Useful for logs and best-effort parsing.
- Fix 3: detect first. Use `chardet` or `charset-normalizer` to guess the encoding from a sample, then open with the detected encoding. Slower but robust.
- Fix 4: read in binary and decode explicitly with an error policy: `path.read_bytes().decode("utf-8", errors="replace")`. Same outcome as Fix 2 but explicit about what's happening.
- Architectural advice for customer-facing imports: detect encoding at upload time, surface a warning to the user if it's not UTF-8, store a normalized UTF-8 version internally. Don't silently re-encode — you lose forensic data.

SCENARIO 03

The For-Else That Wasn't What It Looked Like

A senior dev writes a search loop:

```
for item in inventory:
    if item.matches(criteria):
        result = item
        break
else:
    result = None
```

A new hire on code review asks what the `else` does. The senior says “It’s like an else on the if — if no match found, set result to None.” Several months later, the same new hire writes their own version using `for ... else` but mis-uses it, and the bug ships.

QUESTION

What does Python’s for-else actually do? When does the else block run, when doesn’t it, and why is this construct widely considered a footgun?

EXPECTED DEPTH

- The `else` block runs when the `for` loop completes *without* hitting a `break`. It does *not* run if `break` was triggered.
- So in the senior’s code, `else` runs only if no item matched (no break). If a match was found and `result` was assigned in the loop body, `else` doesn’t run — `result` keeps the value assigned in the loop. The code is correct, but the syntax misleads almost everyone reading it.
- The name “else” is the problem. It reads like “if the for loop didn’t do anything,” but it actually means “if the loop ran to exhaustion.” If you mentally map it to `nobreak:` the construct makes more sense.
- Common bug: someone adds a `return` inside the loop body. That doesn’t count as a `break` — the function returns, the `else` never runs anyway because we left the function. Fine. But if they add an `if early_exit: continue`, the loop still runs to exhaustion, `else` runs. People expect `continue` to behave like `break` for the `else` — it doesn’t.
- Alternative idioms (often clearer):

```
# Using next() with default
result = next((item for item in inventory if item.matches(criteria)), None)

# Or explicit flag
result = None
for item in inventory:
    if item.matches(criteria):
        result = item
        break
```

- Recommendation: avoid `for ... else` in shared codebases unless your team has explicit conventions for it. The cost of a misread is high; the win is a few lines saved.

SCENARIO 04

The Float in the Pricing Engine

An e-commerce backend computes cart totals: `total = sum(item.price * item.quantity for item in cart)`. Prices are stored as Python floats. A customer reports their cart showing \$19.989999999999998 instead of \$19.99. The accountant flags a \$0.01 mismatch between order totals and bank deposits. Multiplied across the year's transactions, the discrepancy is enough to require a manual reconciliation.

QUESTION

Explain the floating-point issue at the bit level. What's the right architecture for money in Python, and why is 'just use Decimal' not the whole answer?

EXPECTED DEPTH

- Python `float` is IEEE 754 double-precision. `0.10` can't be represented exactly in binary — it's an infinite repeating fraction. The stored value is the closest binary approximation, off by $\sim 1e-17$. Multiplications and additions accumulate the rounding error.
- Display layer can hide it (`f"{total:.2f}"` prints `"19.99"`), but the underlying value is wrong. Comparison (`total == 19.99`) fails. Aggregate across millions of transactions — the accountant's discrepancy.
- Fix 1: `decimal.Decimal`. Arbitrary precision, no binary rounding. `Decimal("0.10") + Decimal("0.20") == Decimal("0.30")` — exactly true.

- Why “just use Decimal” isn’t the whole answer:
- Constructor trap: `Decimal(0.1)` — you fed it the inexact float, it faithfully preserves the inexactness. Must use `Decimal("0.1")` or `Decimal(str(0.1))`.
- Boundary trap: any value coming in from JSON, the database, the user form, must be converted from string at the boundary. One stray `float(amount)` anywhere in the pipeline contaminates everything downstream.
- Performance: Decimal is ~30–100× slower than float. For high-volume pricing engines, that matters.
- Library compatibility: numpy doesn’t support Decimal. Pandas does but with caveats. SQLAlchemy stores it correctly but may surprise you on retrieval.
- Better architecture: a `Money` class wrapping `int cents`. Integer arithmetic everywhere internally. `Money(1999)` — always two-cent integer. Type-system enforcement: you can’t pass a `Money` where a float is expected and vice versa. Once it’s baked into the domain model, the whole class of bugs disappears.

SCENARIO 05

The Shared List That Tied Two Users Together

A user-management module has a `User` class. The author wrote:

```
class User:
    permissions = [] # default permissions

    def __init__(self, name):
        self.name = name

    def grant(self, perm):
        self.permissions.append(perm)
```

In a test, `alice = User("Alice"); bob = User("Bob"); alice.grant("read"); print(bob.permissions)` — prints `['read']`. The author intended each user to have their own permissions list. They’re sure they didn’t share anything between Alice and Bob.

QUESTION

What's the mistake, and what's the fundamental Python concept the author misunderstood? How does this relate to the mutable-default-argument bug?

EXPECTED DEPTH

- `permissions = []` at the class level is a *class attribute*, not an instance attribute. There's one list, owned by the class itself, shared by every instance.
- When `alice.grant("read")` calls `self.permissions.append("read")`, it looks up `permissions` on `alice` — not found on the instance, falls back to the class. Mutates the class attribute. `bob.permissions` sees the same list.
- Same root cause as mutable default arguments: “one object created once, used by many.” Python doesn't copy implicitly. If you wrote `permissions = 0`, you'd see the same sharing for reads, but `self.permissions = 5` creates an instance attribute that shadows the class one. With mutable types, you mutate the shared object instead of creating a new attribute.
- Fix: initialize per-instance state in `__init__`.

```
class User:
    def __init__(self, name):
        self.name = name
        self.permissions = []    # fresh list per instance
```

- When IS a class attribute appropriate? Truly shared constants: `DEFAULT_TIMEOUT = 30`. Class-level methods, class-level configuration. Anything immutable and intentionally shared.
- Modern alternative: dataclasses force you to specify `field(default_factory=list)` — the type system nudges you toward correctness:

```
@dataclass
class User:
    name: str
    permissions: list = field(default_factory=list)
```

SCENARIO 06

The Print That Logged Nothing

An ML training script runs for 12 hours on a GPU server. The script prints progress: `print(f"epoch {epoch}, loss {loss}")`. The team launches it with `nohup python train.py > train.log &` and goes home. They come back to a `train.log` that's 0 bytes for 11 hours, then suddenly has all 12 hours of output written at once at the very end. They can't monitor progress in real time.

QUESTION

What's the buffering behavior here? Why is it different when printing to a terminal vs a file, and what are the four ways to fix it?

EXPECTED DEPTH

- Python's `stdout` buffers differently based on whether it's connected to a terminal or a file.
- To a terminal: line-buffered. Each newline flushes. You see output immediately.
- To a file or pipe: block-buffered. Typically 4KB or 8KB before flushing. With redirected output, `print` calls accumulate in memory until the buffer fills — then a chunk lands in the file.
- 12 hours of `epoch N, loss X` lines might not fill the buffer until the very end — or until process exit, when buffers flush as part of cleanup.
- Fix 1: flush manually per write. `print(..., flush=True)`. Simple, explicit.
- Fix 2: reconfigure stdout. `sys.stdout.reconfigure(line_buffering=True)` at the top of the script. All subsequent prints behave as if at a terminal.
- Fix 3: Python flag. `python -u train.py` — the `-u` flag forces unbuffered stdout/stderr. No code changes needed.
- Fix 4: environment variable. `PYTHONUNBUFFERED=1 python train.py`. Same effect as `-u`, configurable via Docker/systemd.
- Best practice for long-running scripts: use the `logging` module instead of `print`. `logging` handles flushing intelligently, supports log levels and rotation, and produces machine-readable timestamps. `print` is for one-off scripts; `logging` is for things you'll run twice.

SCENARIO 07

The Equality That Stopped Being Equal

A trading-system test compares NumPy arrays for equality: `if signals == expected: do_trade()`. Runs fine in unit tests. In production with real data, it raises `ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()`. The author swears they tested with arrays.

QUESTION

Why does `==` work in one context and fail in another? What does NumPy's `__eq__` actually return, and how do you write defensive code that handles both scalars and arrays?

EXPECTED DEPTH

- NumPy overrides `__eq__` to do *elementwise* comparison and return a *new array* of booleans. `array([1,2,3]) == array([1,2,4])` returns `array([True, True, False])`.
- Python's `if` wants a single boolean. NumPy says “the truth of an array isn't defined — do you want any or all true?” and raises rather than guess.
- In unit tests, the arrays were probably length 1 — NumPy *does* allow truth testing on a length-1 array (it returns the single element). So the test passed. Production data with real-length arrays triggers the exception.
- Fix: use NumPy's array-aware operations explicitly.

```
# All elements equal
if np.array_equal(signals, expected):
    do_trade()

# Any element equal
if np.any(signals == expected):
    ...

# Within tolerance (for floats)
if np.allclose(signals, expected, rtol=1e-5):
    ...
```

- Defensive style: never use `==` in a boolean context with values that might be arrays. Always use a function that explicitly reduces to a scalar.
- Same issue applies to Pandas Series, PyTorch tensors, JAX arrays. Any “array-like” type override `__eq__` for elementwise semantics. Library-aware

testing utilities (`numpy.testing.assert_array_equal`) encode the right pattern.

SCENARIO 08

The Dictionary That Grew Unboundedly

A web service caches user lookups in a module-level dict: `cache = {}`. Each request looks up `user_id` in the cache; if missing, queries the database and stores the result. Over weeks, memory usage climbs from 2 GB to 18 GB. The team didn't set a TTL or a size limit — the dict just keeps growing as new users hit the service. Eventually the pod gets OOM-killed and Kubernetes restarts it.

QUESTION

What's the underlying issue, and walk me through the right caching primitives in the Python ecosystem. Why is 'just use a dict' a common mistake?

EXPECTED DEPTH

- A bare dict is an unbounded cache. There's no eviction, no TTL, no max size. It grows monotonically with cache misses.
- For a stable user population, this caps out eventually. For an unbounded keyspace (request IDs, session tokens, anything monotonically increasing), it never caps. Memory leak by design.
- Fix 1: `functools.lru_cache(maxsize=1024)` — stdlib LRU cache decorator. Bounded size, automatic eviction of least-recently-used entries. Use on pure functions where the same input always produces the same output.
- Fix 2: `functools.cache` — Python 3.9+, unbounded version (when you genuinely want no eviction). Use sparingly; usually you want LRU.
- Fix 3: external cache. `cachetools` (rich eviction policies — LRU, LFU, TTL, TLRU). For multi-process: Redis or Memcached. The dict-in-process pattern only works for single-process services.
- Fix 4: `weakref.WeakValueDictionary` — entries can be garbage-collected when no other references exist. Useful when cache entries are also held elsewhere; cache doesn't prevent collection.
- Architectural rule: every cache needs a bounded lifetime, by size or time. "Grows forever, only cleared on restart" is a bug, not a feature.
- Observability: instrument cache hit rate and size. Anything above a few hundred MB or with a hit rate below ~80% deserves a redesign.

SCENARIO 09

The Generator That Was Consumed Twice

A data processing pipeline returns a generator from a database query: `rows = stream_rows(query)`. The caller does some validation and then aggregation: `count = sum(1 for r in rows)` followed by `totals = [r.amount for r in rows]`. The `count` comes out as 50,000 — looks right. The `totals` list is empty. The aggregation code looks completely correct.

QUESTION

What's the bug? Walk me through generator semantics, and explain when you'd want a generator vs a list — and when you'd need to materialize.

EXPECTED DEPTH

- Generators are single-use iterators. Once consumed, they're exhausted — iterating again yields nothing.
- The `sum` iterates through all 50,000 rows, exhausting the generator. The list comprehension then starts iterating again, but there's nothing left. Empty list.
- Why generators are designed this way: streaming. A 50 GB file or 50M-row query result shouldn't fit in memory. The generator yields one row at a time; once it's past a row, it doesn't hold a reference.
- Fix 1: materialize once if you need multiple passes. `rows = list(stream_rows(query))`. Now you can iterate as many times as you want — but you've loaded everything into memory.
- Fix 2: combine the passes. `count, totals = 0, []`; `for r in rows: count += 1; totals.append(r.amount)`. One pass, both results. Constant memory? No, `totals` grows.
- Fix 3: tee. `from itertools import tee; rows_a, rows_b = tee(stream_rows(query))`. Internally buffers the values for the slower consumer. Useful if you can't combine the passes but the dataset fits in memory anyway.
- Fix 4: re-query. If the source is repeatable (a file path, a query function), call it again. `for r in stream_rows(query): ...; for r in stream_rows(query): ...`. Two database round-trips.
- Detection: type annotations help. `def stream_rows(...) -> Iterator[Row]` signals "single pass." `-> list[Row]` signals "materialized, reusable." If the

caller doesn't know which they got, the signature is hiding crucial information.

SCENARIO 10

The Or-Default That Erased Zero

A scientific computing service accepts JSON input including a `temperature` reading. Code does `temp = data.get("temperature") or 20` — “default to room temperature if missing.” Tests pass with various temperatures. One day a researcher reports cryogenic measurements (0°C samples) being silently replaced with 20°C. The team can't see how, since they're using `get` with a default fallback right there.

QUESTION

Why does the `or` fallback misbehave? What's the correct idiom for 'default if missing', and what other falsy values cause similar bugs?

EXPECTED DEPTH

- `x or y` returns `y` if `x` is falsy. `0` is falsy. So when `data["temperature"]` is `0`, the expression returns `20` — silently substituting the default for a legitimate value.
- The pattern conflates “missing” with “falsy.” They're different concepts. `0`, `""`, `[]`, `False` are all falsy but might be valid values.
- Fix: use the second argument of `dict.get` — it's explicitly for default-if-missing.

```
temp = data.get("temperature", 20)
# Returns 20 only if "temperature" is absent.
# Returns 0 if data["temperature"] == 0.
```

- Other falsy-value traps:
- Username: empty string is “user provided empty” — different from “not provided.”
- Quantity: zero items is a valid order line item, not a default fallback.
- Pagination: page 0 might be the first page in a zero-indexed scheme.
- List input: empty list is a valid input meaning “no items selected,” not “use default selection.”

- Defensive idiom: when you mean “if None,” say “if None.”

```
temp = data.get("temperature")
if temp is None:
    temp = 20
```

- Type-annotated approach: use `Optional[float]` and handle the None case explicitly. Type checker keeps you honest. Anywhere you see `x or default` with a numeric or collection type, ask “is 0/empty a valid value here?”

SCENARIO 11

The Late Binding That Captured the Wrong Variable

A team builds a list of callback functions in a loop:

```
callbacks = []
for i in range(5):
    callbacks.append(lambda: print(i))

for c in callbacks:
    c()
```

Expected output: `0 1 2 3 4`. Actual output: `4 4 4 4 4`. The team double-checks the code; the loop variable is right, the lambda captures it. They’re confused.

QUESTION

What’s late binding, why does it bite Python loops with lambdas, and what are the two idiomatic fixes?

EXPECTED DEPTH

- Closures in Python capture variables by *reference*, not by value. The lambda doesn’t snapshot `i`’s value at definition time; it looks up `i` in the enclosing scope at *call* time.
- By the time the callbacks are called, the loop has finished. `i` has its final value, `4`. All five lambdas look up `i` and find `4`.
- Fix 1: default argument as snapshot. Default arguments are evaluated at *definition* time — the opposite of free-variable lookup.

```
callbacks.append(lambda i=i: print(i))
```

- The trick is awkward but idiomatic: the default value of `i` parameter is the current `i` at definition time. When called, `i` inside the lambda refers to the parameter, which is the captured value.
- Fix 2: helper function with explicit closure.

```
def make_callback(value):  
    return lambda: print(value)  
  
for i in range(5):  
    callbacks.append(make_callback(i))
```

- Each call to `make_callback` creates a separate scope with its own `value`. The lambda captures that scope. Different lambdas, different captured values.
- Fix 3: `functools.partial` — binds arguments explicitly at definition time.

```
from functools import partial  
callbacks.append(partial(print, i))
```

- Pattern recognition: anywhere you build callbacks, async tasks, or event handlers in a loop, you face this. Same bug bites with multiprocessing, async tasks, GUI handlers. The fix is always “capture the value, not the variable.”

SCENARIO 12

The Import Cycle That Mostly Worked

A medium-sized codebase has `models.py` and `services.py`. `models.py` does `from services import notify` to send change events. `services.py` does `from models import User` to type-annotate function arguments. Both work in dev. In production under one specific startup order, the app fails with `ImportError: cannot import name 'notify' from partially initialized module 'services'`.

QUESTION

What's happening with the circular import, and how do you fix it without restructuring both files?

EXPECTED DEPTH

- Python's import system runs each module's code top-to-bottom. When `models.py` hits `from services import notify`, Python starts loading `services.py`. `services.py` hits `from models import User` — but `models` is mid-loading, only partially populated. `User` might not be defined yet.
- Python detects the cycle and returns the partial module. `from models import User` looks up `User` on the partial module; if `User` hasn't been defined yet (it's defined later in the file), import fails.
- Why "mostly works": the failure depends on which module is imported first. If `models` is fully loaded before `services` is touched, no cycle. The crash happens only with a specific entry-point ordering.
- Fix 1: import inside the function. `def notify(...): from models import User; ...`. The import runs at call time, by which point both modules are fully loaded. The downside: tiny per-call cost (cached after first call) and the import isn't visible at the top of the file.
- Fix 2: use `TYPE_CHECKING`. For imports that exist only for type hints:

```
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from models import User

def notify(user: "User") -> None:    # string-quoted forward reference
    ...
```

- Imports under `TYPE_CHECKING` run only during static analysis. At runtime they're skipped. The cycle disappears.
- Fix 3: restructure. Pull the shared types into a third module that both depend on. The cycle disappears at the architectural level. This is the cleanest fix when the codebase allows it.
- Underlying principle: circular imports usually signal a layering problem. Library code depending on application code, infrastructure depending on domain. Worth asking whether the cycle reflects a real coupling that should be removed.

SCENARIO 13

The GIL That Made Threading Look Useless

A team has a CPU-bound image processing function. They parallelize with `threading.Thread` — spawn four threads, each processing a quarter of the images. Expected: 4× speedup. Measured: 1.1× speedup, sometimes *slower* than the single-threaded version. The team is told it's the GIL. They want a deeper answer than that.

QUESTION

Walk me through what the GIL actually does, why threading didn't help here, and when threading IS the right answer in Python.

EXPECTED DEPTH

- The Global Interpreter Lock is a mutex in CPython that ensures only one thread executes Python bytecode at a time. Even on a 16-core machine, only one CPython thread is running bytecode at any given instant.
- For CPU-bound pure-Python code, threads don't parallelize. Four threads share the GIL; the OS scheduler hands off the lock between them; total throughput is roughly single-threaded. Plus the synchronization overhead — hence the occasional slowdown.
- When threading helps: I/O-bound work. When a thread is waiting on a network response, file read, database query, it releases the GIL. Other threads run during that wait. For a web scraper hitting 100 URLs, threading is a huge win.
- Also: C extensions can release the GIL while doing CPU work. NumPy operations, image libraries (Pillow), most scientific code, do this. So `numpy.fft.fft` in four threads *can* parallelize — the GIL is released for the C-level math.
- For CPU-bound pure-Python work: use `multiprocessing` or `concurrent.futures.ProcessPoolExecutor`. Each process has its own Python interpreter and its own GIL — true parallelism across CPU cores. Costs more (process startup, IPC overhead, no shared memory by default) but actually scales.
- Modern alternative: `asyncio` for I/O-bound concurrency without thread overhead. Single thread, cooperative scheduling, scales to thousands of concurrent operations.

- Looking forward: Python 3.13 introduced experimental free-threaded mode (no GIL). Will eventually make threading the right answer for CPU-bound code too. Not production-ready yet; ABI compat concerns persist.
- Decision rule: I/O-bound → threading or asyncio. CPU-bound, small data → multiprocessing. CPU-bound, numerical → NumPy/Cython releasing the GIL. Mixed workload → measure.

SCENARIO 14

The Exception That Hid Itself

A long-running data pipeline does database writes inside a try/except: `try: db.write(record); except Exception as e: log.error(e); continue`. The team wants robust processing — if one record fails, keep going. After three months, they notice that errors stop appearing in logs around 2 AM every night, but the pipeline keeps running. Investigation shows the database is actually rejecting many writes around that time, but no log entries exist.

QUESTION

What's going wrong with the exception handling? Walk through the bare-except pattern, and tell me what robust error handling looks like for a long-running pipeline.

EXPECTED DEPTH

- `except Exception` catches everything that inherits from `Exception`. That's nearly all exceptions, but importantly: *not* `KeyboardInterrupt` or `SystemExit` (those inherit from `BaseException`). Useful boundary.
- The 2 AM clue points at a specific class of error. Most likely candidates: `MemoryError`, `ResourceWarning` turning into errors after warning escalation, or a network timeout that takes long enough for the connection to drop and the `log` handler itself to fail.
- If `log.error(e)` itself raises (e.g., log handler can't write to its destination), that exception isn't caught — it propagates out of the except block. But the loop's outer error handler may catch it silently. Or worse, exception during exception handling chains weirdly.
- Better pattern: log the exception *with the traceback*, use `logger.exception` or `exc_info=True` to capture context, and re-raise critical failures.

```
try:
    db.write(record)
except db.DataError as e:          # specific business error – skip the
    record
    logger.warning("Bad record %s: %s", record.id, e)
    bad_records.append(record)
    continue
except db.ConnectionError as e:    # infra error – maybe retry or fail
    loudly
    logger.error("DB connection lost", exc_info=True)
    raise # let supervisor restart us
except Exception as e:            # truly unexpected – log and re-raise
    logger.exception("Unexpected error processing %s", record.id)
    raise
```

- Catch by category, not by net. `except Exception` should be rare. Each except clause should handle a specific known case with a known response.
- Observability beats logs: emit metrics (counters for error types, latency histograms) so the 2 AM dropout shows up as a graph anomaly, not a log diff.
- Test failure scenarios: a robust pipeline test injects errors deliberately. “DB is down” should produce predictable behavior, not silent loss.

SCENARIO 15

The List Comprehension That Used 64 GB

A data engineer writes a deduplication step: `seen_ids = [r.id for r in all_rows]` followed by `if record.id in seen_ids: skip()` in a loop. On a 100M-row dataset, the script runs out of memory — 64 GB peak — and takes 12 hours before crashing. A teammate suggests changing `[]` to `set()`. The script finishes in 4 minutes, peaks at 8 GB.

QUESTION

Walk me through why one character of code change produced 60× faster runtime and 8× less memory. What's the underlying data-structures principle?

EXPECTED DEPTH

- List membership (`x in list`) is $O(n)$ — it scans linearly until found. For 100M rows, each check is up to 100M comparisons. With 100M checks, that's 10^{16} operations. Hours.
- Set membership (`x in set`) is $O(1)$ average — hash lookup. 100M checks, each $O(1)$, total 100M operations. Seconds.
- Memory: a Python list stores object pointers contiguously plus per-object overhead. A set has higher per-element overhead (hash buckets, load factor) *per stored element*, but the algorithm doesn't require keeping a duplicate-tracking structure separate — you build the set once.
- Actually: the 8 GB → 64 GB difference is mostly about the algorithmic blowup creating intermediate state, garbage, and cache thrashing. A linear scan over 100M Python objects produces awful CPU cache behavior and inflates resident memory through fragmentation. The set's contiguous-ish hash table is cache-friendlier.
- Underlying principle: choose data structures by access pattern, not by familiarity. List is the default — ordered, indexed, additive. But for membership testing, set is the right choice every time.
- Pattern recognition: any `if x in collection` inside a loop should make you check whether `collection` is a list or a set. Same line, wildly different complexity.
- Dict lookup is similarly $O(1)$. `user = users_by_id[id]` vs `user = next(u for u in users if u.id == id)` — one is fast, one is linear. The cost of building the dict pays back the first time you use it for more than $O(\log n)$ lookups.
- Process improvement: profile before scaling. The single-machine fix here saved 60× — enough to avoid distributing the job. A lot of “Must use Spark” conversations end with “wait, we used a list for membership checks; switching to a set fixed it.” Big O matters at scale.